Can linear algebra create perfect knockoffs?

Christopher Hemmens[1], Stephan Robert[1]

HEIG-VD, Rue de Galilée 15, 1400 Yverdon-les-Bains, Switzerland

Abstract

As new Model-X knockoff construction techniques are developed, primarily concerned with determining the correct conditional distribution from which to sample, we focus less on deriving the correct multivariate distribution and instead ask if "perfect" knockoffs can be constructed using linear algebra. Using mean absolute correlation between knockoffs and features as a measure of quality, we do produce knockoffs that are pseudo-perfect, however, the optimization algorithm is computationally very expensive. We outline a series of methods to significantly reduce the computation time of the algorithm.

Keywords: Model-X Knockoffs, Linear Algebra

1. Introduction

Candes et al. (2018) asked how to control the false discovery rate in high-dimensional non-linear models, for example, what if you have more features than observations but you know only a handful of the features has any predictive power on the response variable?

The authors' solution was the introduction of Model-X Knockoffs – a set of random variables that statistically mimic the original feature set but, by design, do not have any predictive power on the response variable. The definition of a Model-X Knockoff, $\tilde{X}$, for a feature set X is as follows:

1. for any subset $S \subset \{1, \ldots, p\}$, $(X, \tilde{X})_{\text{swap}(S)} \overset{d}{=} (X, \tilde{X})$;
2. $\tilde{X} \perp\!\!\!\perp Y \mid X$ if there is a response Y.

Note that as long as $\tilde{X}$ is constructed without knowledge of Y, condition 2 is automatically satisfied.

Candes et al. (2018) test this methodology by replicating , who identify gene clusters that predict the development of Crohn's disease in patients. Whereas the original study was able to identify 9 gene clusters, by implementing the Model-X Knockoff methodology, Candes et al. (2018) was able to identify 18 gene clusters.

Since then, several steps have been taken to improve the methodology. For example, Bates et al. (2021) add implementation of the Metropolis-Hastings algorithm (Metropolis et al. (1953), Hastings (1970)) which, although more computationally expensive, improves the "quality" of the knockoffs as measured by the mean absolute correlation between the knockoffs and their features.

The one element that connects every approach to generating knockoffs since the 2018 paper is the focus on deriving the correct conditional distribution from which to sample the knockoffs. But is this a necessary step?

In every case we start with a $p \times n$ matrix and our goal is to generate another $p \times n$ matrix that fulfils a number of linear constraints. Hence, it is possible to reframe this problem not as deriving the correct conditional distribution from which to sample, but a linear algebra optimization problem subject to a series of linear constraints.

In this paper, section 2 summarizes the literature on knockoff construction to date; section 3 outlines how to frame the problem using linear algebra; section 4 tests the methodology on a set of artificially generated random variables with enforced non-zero values of coskewness and cokurtosis; section 5 tests the methodology on a real dataset that in particular presents a serious challenge to the knockoff generation procedure in general; section 6 concludes the paper.

2. Literature Review

In addition to the initial algorithms put forth by Candes et al. (2018), a wide range of knockoff-generating methods have been developed, especially since 2021.

3. Methodology

Suppose we have a multivariate distribution

$$H(x_1, \ldots, x_p) = Pr(X_1 \leq x_1, \ldots, X_p \leq x_p)$$

where the cumulative distribution function (CDF) of X_i is $F_i : \mathbb{R} \to [0, 1]$. Suppose also that we have a sample of this distribution with n observations. Our goal is to use a linear algebra optimization approach to construct perfect Model-X knockoffs. That is, if the original sample is $\mathcal{X} \equiv (X_1, \ldots, X_p) \in \mathbb{R}^{p \times n}$, then we want to find a matrix $\tilde{\mathcal{X}} \equiv (\tilde{X}_1, \ldots, \tilde{X}_p) \in \mathbb{R}^{p \times n}$ such that the statistical moments of each feature and the other features are equal to the statistical moments of that feature's knockoff and the other features, as well as between that feature's knockoff and the other features' knockoffs, while minimizing the absolute value of the correlation between each feature and its knockoff.

We focus on equating up to the fourth statistical moment: correlation, coskewness, and cokurtosis. To understand our approach, you have to know that generating knockoffs using linear algebra is disproportionately computationally expensive compared to sampling from a distribution and, as a result, our methodology concentrates on minimizing the amount of work a computer needs to do.

3.1. Setting Constraints

We define our (standardized) (m, k)-moment between variables X and Y to be

$$\mu_{m,k}(X, Y) = \frac{\frac{1}{n} \sum_{i=1}^{n} \left((x_i - \mu_1(X))^k (y_i - \mu_1(Y))^{m-k} \right)}{(\mu_2(X))^{k/2} (\mu_2(Y))^{(m-k)/2}}$$

where

$$\mu_1(X) = \frac{1}{n} \sum_{i=1}^{n} x_i$$

and

$$\mu_2(X) = \frac{1}{n} \sum_{i=1}^{n} (x_i - \mu_1(X))^2$$

This means our first constraints on each knockoff are that they have the same distributions as their features. We ensure this by determining the distribution of the feature

in advance, and then performing a Kolmogorov-Smirnov test on the knockoff. We set the constraint that the p-value of the test's result is as close to 1 as possible.

The next set of constraints are simply that for every value of $i, j = 1, \ldots, p$ where $i \neq j$, and $2 \leq m \leq 4$ with $1 \leq k \leq (m-1)$, we have

1. $\mu_{m,k}(\tilde{X}_i, X_j) = \mu_{m,k}(X_i, X_j)$
2. $\mu_{m,k}(X_i, \tilde{X}_j) = \mu_{m,k}(X_i, X_j)$
3. $\mu_{m,k}(\tilde{X}_i, \tilde{X}_j) = \mu_{m,k}(X_i, X_j)$

We are careful not to duplicate any of these constraints, for example, $\mu_{4,1}\left(X_i, \tilde{X}_j\right)$ and $\mu_{4,3}\left(\tilde{X}_j, X_i\right)$ are the same condition, so we only include it once in the optimization process.

3.2. Optimizing

Now the constraints are set, we want to optimize the knockoffs such that the correlations between the features and their knockoffs are as close to zero as possible. In many circumstances, it may not be possible to generate knockoffs with zero correlation with their features, because the resultant multivariate distribution $(X, \tilde{X})$ still needs to have a covariance matrix that is positive-semidefinite in order to be a valid distribution.

For this reason, we set the minimization value to be the sum of the squared correlations between the features and their knockoffs. Formally,

$$\sum_{i=1}^{p} \left(\mu_{2,1}\left(X_i, \tilde{X}_i\right)\right)^2 \tag{1}$$

By performing the minimization in this way, we trend all the values to zero while also ensuring that the minimization of absolute correlations far from zero are prioritised over those already close to zero.

3.3. Initial Guess

Since this approach is computationally expensive, we want to start as close to a viable solution as possible before running the optimization. That way, if a single iteration of the optimization takes an excessively long time, we can reduce the parameter

indicating the maximum number of iterations and be comfortable that we started with something approximating a viable knockoff.

In this paper, we approximate $H(\cdot)$ using the Gaussian copula:

$$H(x_1, \ldots, x_p) \approx C(x_1, \ldots, x_p) \equiv \Phi\left(\Phi^{-1}(F_1(x_1)), \ldots, \Phi^{-1}(F_p(x_p))\right)$$

This approximation allows use to generate an initial guess for the knockoffs that already minimizes (1) somewhat while matching the covariance constraints using the formula provided in Section 3.1.1 of Candes et al. (2018).

All that remains is to set the model's tolerance, maximum iterations, and learning rate.

4. Speed Test Results

To test the efficiency of the algorithm, we randomly generate n observations for p independent normal random variables with mean 0 and variance 1. We then shuffle the elements to artificially create non-zero covariance, coskewness, and cokurtosis among the variables. For variable X_i,

- if $\text{mod}(i, 4) = 1$, we weight smaller observations towards the end of the sample,
- if $\text{mod}(i, 4) = 2$, we weight larger observations towards the end of the sample,
- if $\text{mod}(i, 4) = 3$, we weight observations close to the mean towards the end of the sample,
- if $\text{mod}(i, 4) = 0$, the sample is left unchanged.

We run the algorithm using a CPU 11th Gen Intel(R) Core(TM) i7-11700F @ 2.50GHz with 16GB RAM running on Windows 11 Home 64-bit OS.

<u>$p = 4$</u>

Tables 1 and 2 show how the summary statistics of the knockoffs evolve as more iterations in the knockoff construction process are performed, in addition to the correlation between each feature and its knockoff, which we want to be as close to zero possible.

Knockoff	Iterations	Mean		Variance		Dist. p-value	
	$n =$	100	1000	100	1000	100	1000
$\tilde{X}_1$	Target	-0.1258	-0.0020	0.8988	0.9247	100%	100%
	3	-0.1200	-0.0070	0.9438	0.9440	60.4%	42.0%
	10	-0.1252	-0.0021	0.9044	0.9244	98.6%	56.0%
	20	-0.1261	-0.0021	0.8995	0.9247	99.2%	61.1%
$\tilde{X}_2$	Target	0.0214	-0.0529	0.8296	0.9997		
	3	0.0193	-0.0534	0.9726	1.0224	75.8%	10.7%
	10	0.0212	-0.0529	0.8437	1.0019	65.0%	6.0%
	20	0.0214	-0.0529	0.8261	0.9995	76.5%	12.9%
$\tilde{X}_3$	Target	0.0349	-0.0494	0.8706	1.0176		
	3	0.0108	-0.0561	0.9544	1.0388	89.1%	70.3%
	10	0.0342	-0.0494	0.8729	1.0192	50.1%	21.5%
	20	0.0348	-0.0494	0.8715	1.0175	81.9%	30.5%
$\tilde{X}_4$	Target	-0.0392	0.0327	0.9514	0.9751		
	3	-0.0609	0.0309	1.0852	0.9778	59.2%	91.5%
	10	-0.0398	0.0327	0.9717	0.9757	42.4%	93.7%
	20	-0.0399	0.0327	0.9594	0.9751	35.8%	95.2%

Table 1: Summary statistics for the knockoffs when $p = 4$

Because of the way we construct our initial guess, these values are already very close to what we want the final result to be, so they don't vary much as the optimization process continues. The important question is whether the constraints on the moments are fulfilled and, if so, how quickly and how precisely.

With four features, we have six relationships whose knockoff moments need to match those of their features. Figures 1, 2, and 3 show the six relationships across the six moments of interest: correlation, left and right coskewness, and left, central, and right cokurtosis, and their evolution as we perform additional iterations in the optimization process.

Knockoff	Iterations	Correlation with Feature	
	$n =$	100	1000
$\tilde{X}_1$	Initial Guess	0.047	0.009
	3	0.121	0.058
	10	0.147	0.058
	20	0.105	0.057
$\tilde{X}_2$	Initial Guess	-0.105	-0.013
	3	-0.032	0.037
	10	0.021	0.078
	20	0.013	0.080
$\tilde{X}_3$	Initial Guess	0.022	-0.030
	3	0.031	-0.026
	10	0.016	-0.023
	20	0.011	-0.021
$\tilde{X}_4$	Initial Guess	-0.026	-0.009
	3	0.036	-0.013
	10	0.228	-0.015
	20	0.185	-0.013

Table 2: Correlation of each feature and its knockoff when $p = 4$

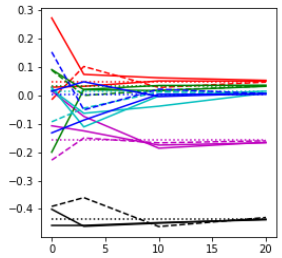


Figure 1: Evolution of correlation constraint adherence over 20 iterations, $p = 4$, $n = 100$

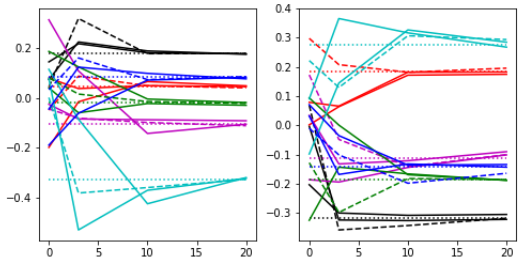


Figure 2: Evolution of coskewness constraint adherence over 20 iterations, $p = 4$, $n = 100$

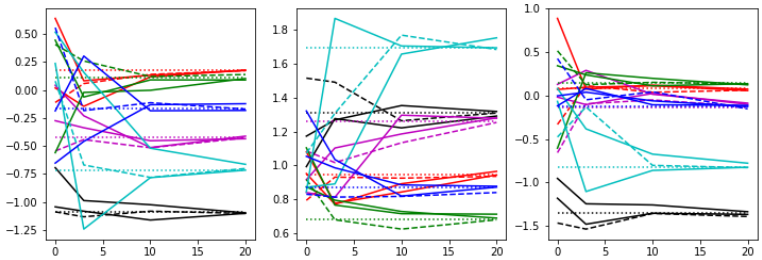


Figure 3: Evolution of cokurtosis constraint adherence over 20 iterations, $p = 4$, $n = 100$

Each colour represents a different relationship, for example, black shows the relationship between the first and second features, while green shows the relationship between the second and third features. The dotted line shows the real value of the relationship and therefore the target value of the constraint. The two solid lines show the value of the moment between one of the features and the knockoff of the other feature,

Total Observations			Iterations		
n	p	$n \times p$	3	10	20
100	4	400	0:00:07	0:00:13	0:00:26
	8	800	0:00:26	0:00:52	0:01:27
1000	4	4000	0:01:37	0:06:21	0:12:40
	8	8000	0:10:03	0:41:37	1:26:15

Table 3: Time taken to complete optimization for a set number of iterations in H:M:S

while the dashed line shows the value of the moment between the two knockoffs.

We can see that the optimization process is reaching the target moments quickly and precisely with this small dataset so we can be confident that the technique is working and producing close to pseudo-perfect knockoffs.

Figure 4 shows that if we increase the number of features, we see some degradation in the speed with which the constraints converge. In this context, it's important to note that going from 4 features to 8 increases the number of relationships from 6 to 28. In the current code, all of the cokurtosis constraints, for example, are entered into the model as a single constraint. The same is true for coskewness and correlation. This is to make the code as computationally efficient as possible, however, we could make the constraints for left coskewness and right coskewness separate constraints to make them converge sooner. The question is whether the increase in speed of convergence is adequately balanced with the loss of computational efficiency. In Subfigures 4a and 4b, where $p = 4$, we show all the relationships. In 4c and 4d, where $p = 8$, we just show the quintiles for the sake of readability.

In terms of speed, Table 3 shows how long each optimization process took. The code is already optimized to scale well with the number of features, p. We hope to find additional efficiency savings for the total number of observations.

5. Real Data Results

To test the effectiveness of this approach on real data, we use daily returns from the constituents of the Swiss Market Index (SMI) from the 26th May 2017 until the 8th April 2019. This is 481 observations from 20 features.

The daily returns of these 20 stocks are scaled to have mean 0 and variance 1 and the scaled dataset is fit to a Gaussian copula. All univariate variables are modelled as Student-t distributions for which the degrees of freedom range from 3.08 to 5.81.

Using the copula, we generate the initial guess and perform the optimisation procedure as outlined in Section 3. Since there are 20 features, we have 190 relationships and 1140 moment-based constraints. Table 4 shows the aggregated constraints over a given number of iterations. The target for each constraint is zero.

Moment	Iterations	Constraint Value
Correlation	Initial Guess	0.825
	20	0.049
	100	0.006
Coskewness	Initial Guess	38.106
	20	1.031
	100	0.090
Cokurtosis	Initial Guess	1692.465
	20	22.992
	100	1.757

Table 4: Evolution of constraint values where 0 is the target

The time to complete 100 iterations was 13:37:27. The goal of the procedure is to produce knockoffs whose correlations with their features are as close to zero as possible whilst adhering to the moment constraints, and Table 5 shows the quartiles of the correlations achieved.

Table shows the results for Lonza and UBS, which got the lowest and highest resultant correlations respectively, as well as Nestle, which is a big company that is representative of the middle of the distribution.

Quartile	Min	0.25	0.5	0.75	Max
Correlation	0.36	0.51	0.60	0.71	0.92

Table 5: Minimization results for correlation between knockoffs and features

Stock	Dist. p-value	Correlation
Lonza	41.5%	0.357
Nestle	89.5%	0.527
UBS	52.0%	0.919

Table 6: Fit of the knockoffs to the right univariate distributions and their correlation with their features

6. Discussion

If one uses mean absolute correlation in order to determine the "quality" of a knockoff, then it seems that this algorithm is indeed capable of generating "perfect" knockoffs. The question is how much time is required in order to generate them.

By using the Gaussian copula to estimate the multivariate distribution of the features, we're able to generate an initial guess for the model in a matter of seconds that not only gives us knockoffs whose distributions already match those of the features, but also already satisfy to some extent the correlation requirements of good-quality knockoffs.

With this initial guess, the minimization algorithm doesn't need to run for that many iterations in order align the knockoffs with correct multivariate distribution as well as the constraints of the higher-order moments: namely coskewness and cokurtosis.

As an initial step, this is a very promising technique for generating Model-X knockoffs, but the nature of the optimization algorithm suggests that there is lots of scope to significantly improve the computation cost of this approach, thereby significantly increasing the accessibility to "perfect" knockoffs for a wider range of problems.

		Nestle			UBS		
		Real	Feature	Knockoff	Real	Feature	Knockoff
Lonza	Correlation	0.364	0.366	0.365	0.424	0.424	0.424
	Left Coskewness	-0.030	-0.026	-0.030	-0.122	-0.122	-0.120
	Right Coskewness	-0.155	-0.157	-0.157	-0.273	-0.274	-0.275
	Left Cokurtosis	1.181	1.178	1.179	1.642	1.643	1.643
	Central Cokurtosis	1.579	1.577	1.574	1.871	1.872	1.875
	Right Cokurtosis	1.417	1.432	1.420	1.811	1.805	1.807
Nestle	Correlation				0.337	0.336	0.336
	Left Coskewness				-0.136	-0.134	-0.136
	Right Coskewness				-0.233	-0.237	-0.236
	Left Cokurtosis				1.599	1.596	1.605
	Central Cokurtosis				1.793	1.796	1.786
	Right Cokurtosis				1.437	1.434	1.444

References

E. Candes, Y. Fan, L. Janson, J. Lv, Panning for gold, Journal of the Royal Statistical Society. Series B (Statistical Methodology) 80 (2018) 551–577.

Y. Romano, M. Sesia, E. Candès, Deep knockoffs, Journal of the American Statistical Association 115 (2020) 1861–1872.

S. Bates, E. Candès, L. Janson, W. Wang, Metropolized knockoff sampling, Journal of the American Statistical Association 116 (2021) 1413–1427.

N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, E. Teller, Equation of state calculations by fast computing machines, The journal of chemical physics 21 (1953) 1087–1092.

W. K. Hastings, Monte carlo sampling methods using markov chains and their applications (1970).

Z. Ren, Y. Wei, E. Candès, Derandomizing knockoffs, Journal of the American Statistical Association (2021) 1–11.

M. Sklar, Fonctions de répartition à n dimensions et leurs marges, in: Annales de l'ISUP, volume 8, 1959, pp. 229–231.

R. B. Nelsen, An introduction to copulas, Springer science & business media, 2007.

K. Aas, C. Czado, A. Frigessi, H. Bakken, Pair-copula constructions of multiple dependence, Insurance: Mathematics and economics 44 (2009) 182–198.

A. R. Vásquez, J. U. Márquez Urbina, G. González Farías, G. Escarela, Controlling the false discovery rate by a latent gaussian copula knockoff procedure, Computational Statistics (2023) 1–24.

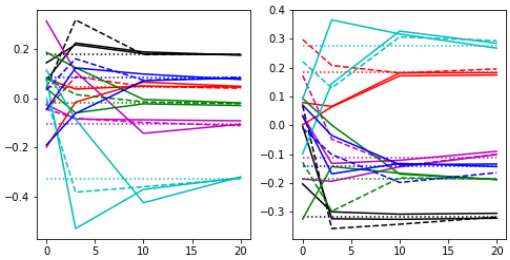
P. Berti, E. Dreassi, F. Leisen, L. Pratelli, P. Rigo, New perspectives on knockoffs construction, Journal of Statistical Planning and Inference 223 (2023) 1–14.

P. Embrechts, F. Lindskog, A. McNeil, Modelling dependence with copulas, Rapport technique, Département de mathématiques, Institut Fédéral de Technologie de Zurich, Zurich 14 (2001) 1–50.

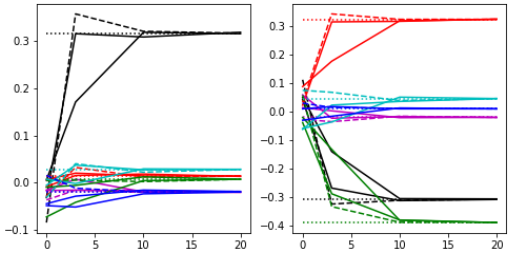
J. B. Heaton, N. G. Polson, J. H. Witte, Deep learning for finance: deep portfolios, Applied Stochastic Models in Business and Industry 33 (2017) 3–12.

S. Kim, S. Kim, Index tracking through deep latent representation learning, Quantitative Finance 20 (2020) 639–652.

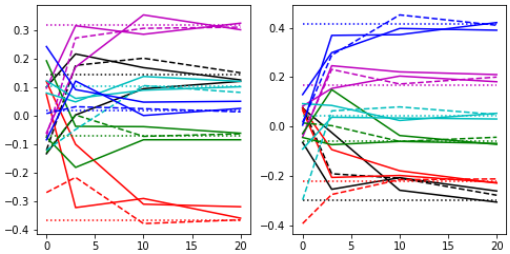
C. Zhang, S. Liang, F. Lyu, L. Fang, Stock-index tracking optimization using autoencoders, Frontiers in Physics 8 (2020) 388.



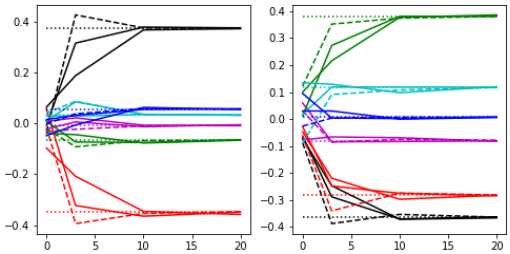
(a) Coskewness Constraint, $p = 4$, $n = 100$



(b) Coskewness Constraint, $p = 4$, $n = 1000$



(c) Coskewness Constraint, $p = 8$, $n = 100$



(d) Coskewness Constraint, $p = 8$, $n = 1000$

Figure 4: Constraint convergence with increasing numbers of features and observations